

5

Securing Your Spring Boot Applications

Welcome to an important stage in your Spring Boot learning journey. In this chapter, we focus on security: a crucial aspect that will help you protect your applications against evolving digital threats. Here, you'll learn how to implement strong security with Spring Boot 3.0, which includes techniques using **Open Authorization 2.0 (OAuth2)**, **JSON Web Token (JWT)**, and **Role-Based Access Control (RBAC)**. We shall also go into the details of how to secure a reactive application.

You will learn how to authenticate users using OAuth2 and manage secure tokens using JWT. You will also master RBAC, whose job is to provide the right access to the right users. We even have a dedicated section just for reactive developers who want to be assured that their reactive apps are at least as secure as their standard web apps.

Why does this matter? In our digital world, security is not a feature; it's a way of life. The concepts you are going to be a master of with this will help you to craft applications that are safe and trustworthy, protecting your data and the identity of users.

In the end, you will have a secure, running sample application that implements all the security principles mentioned. Ready to make your applications safe and sound? Let's dig in!

In this chapter, we'll cover the following topics:

- Introducing security in Spring Boot 3.0
- Implementing OAuth2 and JWT
- Implementing RBAC in Spring Boot
- Securing reactive applications

Let's begin this journey to make your Spring Boot applications secure and robust!

Technical requirements

For this chapter, our local machines will need the following:

- **Java Development Kit (JDK) 17**
- A modern **Integrated Development Environment (IDE)**; I recommend IntelliJ IDEA
- **GitHub repository:** You can clone all repositories related to *Chapter 5* from here: <https://github.com/PacktPublishing/Mastering-Spring-Boot-3.0>
- Docker Desktop

Introducing security in Spring Boot 3.0

In this chapter, we will delve into the aspect of security in Spring Boot 3.0. Security is not a checkbox; it is an important ingredient for building any application. Here, we will cover security features that come integrated out of the box and are provided by Spring Boot, to secure our application from scratch. Now, let's have a look at how all these features can be customized and extended to best serve our needs, ensuring that we implement not only a functional application but also a secure one.

Firstly, let's explore Spring Boot's security architecture, which has been built to be robust and flexible. You will see how Spring Boot makes it easy to secure your application, with some sensible defaults out of the box, but also the ability to customize them for more advanced use cases. By the end of this chapter, you will realize why security is so important, and what tools Spring Boot 3.0 provides to implement effective security.

Exploring Spring Boot 3.0's security features

As we're starting to build any kind of web application, the very first thing that comes to mind is to ensure that they are secure. Here, Spring Boot 3.0 comes with all the powerful utilities to secure our applications. In this section, we will dig into how we make a security architecture in Spring Boot 3.0 that helps us achieve that, ensuring that our web apps are safe and sound.

The best thing about Spring Boot is that it's really easy to set up security. It is based on Spring Security, a framework for securing everything. Think of Spring Security as a vigilant security guard who checks for every ID at the door; this way, it ensures that access to particular parts of your application is granted to only those who have the right permissions.

The following are some of the most important security features of Spring Boot 3.0:

- **Configuring security in our application:** Setting up the security for your application with Spring Boot 3.0 is like configuring settings on your mobile device. You get to decide what you want on and what to turn off. Spring Boot allows us to very easily define who can access what in our application. We do this with a simple yet powerful configuration within our code.
- **Authenticating users:** At its most basic, authentication is how we verify who a person is. Spring Boot 3.0 helps with this. It can be through a username and password, through tokens, or in some other way, and Spring Security is there to help you. It is very much like having a bouncer at the door of your app, making sure only authorized users are able to get in.
- **Authorization:** Upon identification of who a person is, one has to define what the person is allowed to do. Spring Security lets us set up rules for what authenticated users are allowed to access. It's like giving out keys to different doors in your app, based on who needs to go where. As you can see, while authentication is verifying the identity of the user, authorization determines what resources and actions the user is permitted.
- **Defense against common threats:** The internet can be like a jungle with different kinds of threats hiding in it. The security architecture of Spring Boot 3.0 is designed to protect against these threats. From **Cross-Site Scripting (XSS)** to SQL injection, Spring Security helps protect your application against commonly known vulnerabilities.
- **Leveraging advanced security features:** Getting deeper into it, Spring Boot 3.0 provides yet another advanced security feature set. It adds OAuth2 for securing access to APIs and **JSON Web Tokens (JWTs)** for stateless authentication. It's like adding an advanced security system to your application, like cameras and detectors.

By now, we have understood and applied the security architecture of Spring Boot 3.0, and we are in a position to develop robust and secure applications. We have seen the basics—from authentication and authorization to protecting against threats, including advanced features. It's all about ensuring that our applications are safe places for our users to visit and engage with.

This section showed that Spring Boot 3.0 has a strong set of features under its security domain, built to keep our applications safe. Remember that security is an ongoing process. Continuously monitoring and updating is the way to go. The steps of continuously monitoring and updating our measures for security are described in this chapter. This approach ensures our applications remain secure over time, adapting to new challenges as they arise.

In this section, we have learned about Spring Boot 3.0's basic security features. In the next section, we will start to look at the code for implementing Spring Boot security.

Setting up a basic security configuration

Let's walk through setting up a basic security configuration, ensuring your app is protected right from the start. The following guided walk-through will show you how to introduce the security layer to our sample book store application, making it a safe space for users and data alike:

1. **Adding Spring security dependencies:** First up, we need to add Spring Security to our project. As we're using Gradle, we should include the following in our `build.gradle` file:

```
implementation 'org.springframework.boot:spring-boot-starter-security'
```

Now we have added all the necessary libraries to our project.

2. **Configuring web security:** Next, we'll create a basic security configuration class:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity
http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(session -> session.
sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(authz -> authz
                .requestMatchers("/login").permitAll()
                .anyRequest().authenticated()
            )
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }
}
```

This code blocks all requests except the `/login` endpoint in our current book store implementation as we don't have that endpoint yet. So, all other endpoints will be authenticated by this configuration. You can check it by executing a `curl` request like this:

```
curl -v GET --location "http://localhost:8080/books"
```

You will get a DENY response from this request because we have hidden the endpoints behind Spring Security.

Let's understand what we have introduced with this `SecurityConfig.java` class:

- `@Configuration`: This annotation marks the class as a source of Bean definitions for the application context. It tells Spring that this class contains configuration information.
- `@EnableWebSecurity`: This annotation enables Spring Security's web security support and provides the Spring MVC integration. It signals to the Spring Framework to start adding security configurations.
- `@Bean`: This annotation tells Spring that the method will return an object that should be registered as a bean in the Spring application context. In this case, it's the `SecurityFilterChain` bean.
- `public SecurityFilterChain securityFilterChain(HttpSecurity http)`: This method defines the security filter chain. It takes an instance of `HttpSecurity` as a parameter, allowing you to configure web-based security for specific HTTP requests.
- `.csrf(AbstractHttpConfigurer::disable)`: **Cross-Site Request Forgery (CSRF)** protection is disabled in this configuration. CSRF protection is a security feature that prevents malicious websites from performing unauthorized actions on behalf of authenticated users. For this reason, disabling CSRF may be appropriate for APIs or applications that are only accessed by other applications, not by browsers directly.
- `.sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))`: This configures the session creation policy to be stateless. In a stateless API, no session information is stored on the server between requests. This is typical for REST APIs, where each request is independent and authentication is done via tokens, not cookies.
- `.authorizeHttpRequests(authz -> authz)`: This part starts the authorization configuration.
- `.requestMatchers("/login").permitAll()`: This line specifies that requests matching the `/login` pattern should be allowed without authentication. It's a way to define public endpoints within your application.
- `.anyRequest().authenticated()`: This ensures that any request not matched by previous matchers must be authenticated. It's a catch-all that secures the rest of the application by default.
- `.httpBasic(Customizer.withDefaults())`: Enables HTTP Basic authentication. This is a simple, stateless authentication mechanism that allows a client to send a username and password with each request. The `Customizer.withDefaults()` part applies default configurations for HTTP Basic, making setup straightforward.

This code basically sets up a security filter chain that in effect disables CSRF protection and is perfect for stateless applications. It provides for the REST API's required stateless session management and allows access only on a few URLs (such as through `/login`) as public. For all other URLs, it enforces to authenticate thorough authentication via HTTP Basic authentication.

In this section, you have learned how to put a basic security setup in place for your Spring Boot application. But our investigation into security is not yet complete.

By knowing and implementing these basic security configurations, you are making serious steps toward the creation of secure and trusted applications. Always keep in mind that a secure application does not only mean the protection of data but trusting your users in relation to the application.

Implementing OAuth2 and JWT

Moving on with the topic of application security, we will now discuss some of the more advanced mechanisms that cater to a changing environment. This brings us to two important technologies: OAuth2 and JWT. Both of them are critical players in improving security configurations for modern applications; however, both have different roles and complement each other to achieve the overall bigger picture of secure authentication and authorization.

In subsequent sections, we provide details of how to set up OAuth2 for Keycloak. We detail the configuration of OAuth2 for Keycloak, followed by the required code snippets. We will use Keycloak, an open source platform with full support for OAuth2 off-the-shelf protocols and extensive abilities to be customized, to provide **Identity and Access Management (IAM)**.

Configuring OAuth2 with Keycloak

Going a step further into the realm of advanced security, one of the key steps in increasing the security of your application is the configuration of OAuth2. We are going to use Keycloak for IAM. We chose Keycloak because it is open source, and its setup process is very easy. It is a tool to simplify complexities in the security process with regard to our applications. It includes built-in support for OAuth2, therefore making everything that pertains to the management of user identity and the protection of user access to your applications easier. Think of Keycloak as a kind of gatekeeper who already knows your users well and ensures that only those who have the right permission can access certain parts of your application.

Let's start our step-by-step implementation. We will use Docker Compose to run Keycloak besides our PostgreSQL and MongoDB setup together. We will update our current `docker-compose.yml` file as follows. You can also find it in the GitHub repository at <https://github.com/PacktPublishing/Mastering-Spring-Boot-3.0/blob/main/Chapter-5-implementing-oauth2-jwt/docker-compose.yml>:

```
version: '3.1'
services:
  db:
    image: postgres
    restart: always
    environment:
      POSTGRES_PASSWORD: yourpassword
      POSTGRES_DB: bookstore
    ports:
      - "5432:5432"
  mongodb:
    image: mongo
    restart: always
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_DATABASE: bookstore
  keycloak_db:
    image: postgres
    restart: always
    environment:
      POSTGRES_DB: keycloak
      POSTGRES_USER: keycloak
      POSTGRES_PASSWORD: keycloakpassword
    ports:
      - "5433:5432"
  keycloak:
    image: bitnami/keycloak:latest
    restart: always
    environment:
```

```
KEYCLOAK_USER: admin
KEYCLOAK_PASSWORD: admin
DB_VENDOR: POSTGRES
DB_ADDR: keycloak_db
DB_PORT: 5432
DB_DATABASE: keycloak
DB_USER: keycloak
DB_PASSWORD: keycloakpassword
ports:
  - "8180:8080"
depends_on:
  - keycloak_db
```

We kept our MongoDB and PostgreSQL setup as it is and introduced two new images: `keycloak_db` and `keycloak`. Let's break down the parameters here:

- **Keycloak database:**

- `image: postgres`: Specifies the Docker image to use for the container. In this case, it's using the official PostgreSQL image.
- `restart: always`: This setting ensures the container always restarts if it stops. If Docker restarts or the container exits for any reason, this setting will cause it to be restarted.
- `environment`: Defines environment variables for the container. For `keycloak_db`, it sets the PostgreSQL database (`POSTGRES_DB`) to `keycloak`, the database user (`POSTGRES_USER`) to `keycloak`, and the user's password (`POSTGRES_PASSWORD`) to `keycloakpassword`.
- `ports`: Maps ports from the container to the host machine. `"5433:5432"` maps the default PostgreSQL port inside the container (5432) to port 5433 on the host. This allows you to connect to the database from the host machine using port 5433. We are using 5433 because we have already used 5432 in the PostgreSQL database of our application.

- **Keycloak service:**

- `image: bitnami/keycloak:latest`: Specifies the Docker image for the Keycloak server, using the latest version of the `bitnami/keycloak` image.
- `restart: always`: Similar to `keycloak_db`, ensures the Keycloak container is always restarted if it stops for any reason.

- `environment`: Sets environment variables specific to the Keycloak server:
 - `KEYCLOAK_USER`: The admin username for Keycloak (admin).
 - `KEYCLOAK_PASSWORD`: The admin password for Keycloak (admin).
 - `DB_VENDOR`: Specifies the type of database being used (POSTGRES in this case).
 - `DB_ADDR`: The address of the database container. Using the `keycloak_db` service name allows Keycloak to find the database within the Docker network.
 - `DB_PORT`: The port on which the database is listening (5432).
 - `DB_DATABASE`: The name of the database Keycloak should use (keycloak).
 - `DB_USER` and `DB_PASSWORD`: The credentials Keycloak will use to connect to the database.
- `ports`: Maps the Keycloak server port from the container to the host machine. "8180:8080" maps the internal port 8080 (Keycloak's default port) to 8180 on the host. This allows you to access the Keycloak server from the host machine using port 8180. We have changed the original port because our Spring Boot application is using port 8080.
- `depends_on`:
 - `keycloak_db`: Specifies that the `keycloak` service depends on the `keycloak_db` service. Docker Compose will ensure that `keycloak_db` is started before `keycloak`.

This setup provides a robust and straightforward way to deploy Keycloak with a PostgreSQL database using Docker Compose. By understanding these parameters, you can customize your setup to fit your specific needs, such as changing ports, database names, or credentials.

When we run the `docker-compose up` command in our terminal in the directory of this `docker-compose.yml` file, our four services (PostgreSQL, MongoDB, PostgreSQL for Keycloak, and the Keycloak service) will be up and running in our local machine. In the next step, we will configure the Keycloak server according to our needs.

Configuring the Keycloak service

As we defined in the Docker Compose file, we now have the URL and port of the Keycloak server. Open a web browser and navigate to `http://localhost:8180/`. Log in with the admin credentials we set earlier in the Docker Compose file to access the Keycloak administration console. In our example, the username is `admin` and the password is `admin`. With the following steps, let us configure a Keycloak service:

1. Creating a realm:
 - I. Click on **Add realm** and name it `BookStoreRealm`.

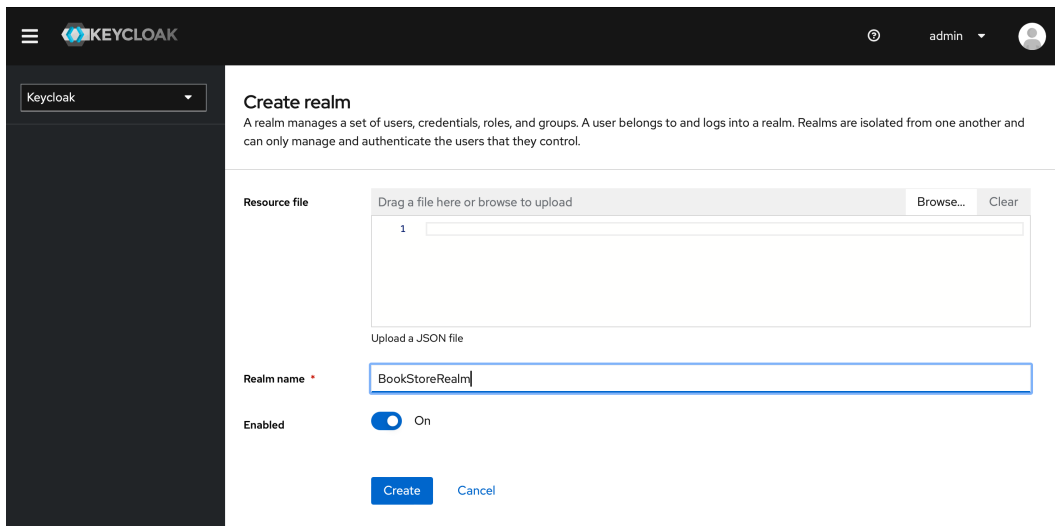


Figure 5.1: Add realm screen

- II. Click **Create**.
2. Creating a client:
 - I. Inside your realm, navigate to **Clients** and click **Create**.
 - II. Set **Client ID** to `bookstore-client` and **Root URL** to the URL of your Spring Boot application (for our application, it is `http://localhost:8080`).

Figure 5.2: Client create screen

- III. On the next screen, set client authentication to true and click the save button at the end of the client creation flow.
- IV. Under **Credentials**, note the secret as you will need it for your application properties in the Spring Boot application in the *Configuring the book store application for OAuth2* section.

Figure 5.3: Credentials of the client screen

3. Creating a user:

Navigate to **Users**, add a user, and set up a password under the **Credentials** tab.

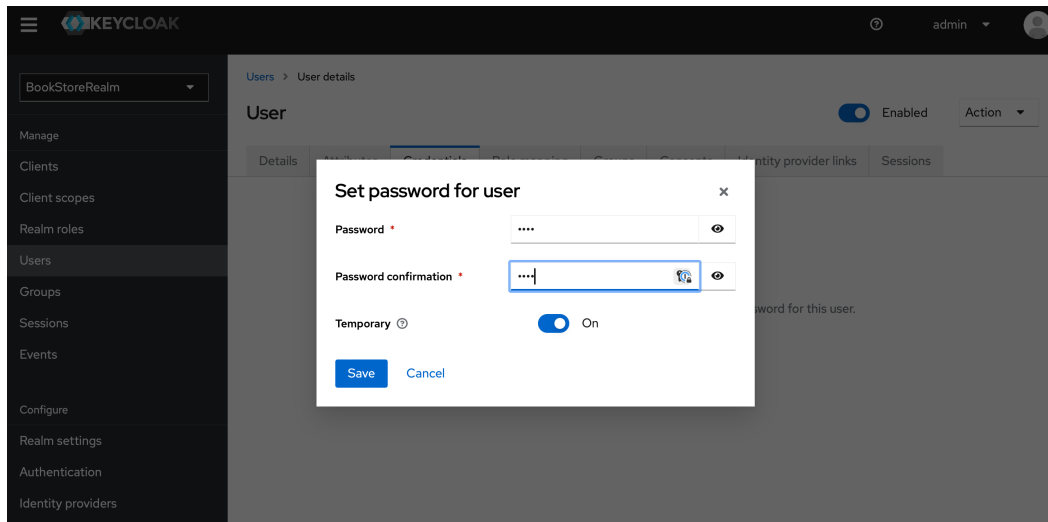


Figure 5.4: User Credentials screen

Now, we will have a user with a username and password and the secret of our Keycloak client, which will be used in the next section.

Important note

Keycloak regularly undergoes updates and improvements, which may result in changes to the user interface. As a result, the UI might look different from the descriptions and screenshots provided in this chapter.

Configuring the book store application for OAuth2

We have a Keycloak server and it has been configured; now, we will need to configure our book store application so it can communicate with the Keycloak server:

1. **Add dependencies:** Ensure your Spring Boot project has the following dependencies in the `build.gradle` file:

```
implementation 'org.springframework.boot:spring-boot-starter-  
oauth2-client'  
implementation 'org.springframework.boot:spring-boot-starter-  
oauth2-resource-server'
```

2. **Configure application.properties:** Add the following properties to your application.properties file, replacing placeholders with your actual Keycloak and client details:

```
spring.security.oauth2.client.registration.keycloak.client-id=bookstore-client
spring.security.oauth2.client.registration.keycloak.client-secret=<Your-Client-Secret>
spring.security.oauth2.client.registration.keycloak.client-name=Keycloak
spring.security.oauth2.client.registration.keycloak.provider=keycloak
spring.security.oauth2.client.registration.keycloak.scope=openid,profile,email
spring.security.oauth2.client.registration.keycloak.authorization-grant-type=authorization_code
spring.security.oauth2.client.registration.keycloak.redirect-uri={baseUrl}/login/oauth2/code/keycloak
spring.security.oauth2.client.provider.keycloak.issuer-uri=http://localhost:8180/auth/realms/BookStoreRealm
spring.security.oauth2.resourceserver.jwt.issuer-uri=http://localhost:8180/auth/realms/BookStoreRealm
```

These settings are used to configure OAuth2 client registration and resource server properties for a Spring Boot application. They specifically configure the application to use Keycloak as the authentication provider. Let's break down what each setting means:

- `spring.security.oauth2.client.registration.keycloak.client-id`: This is the unique identifier for the OAuth2 client registered in Keycloak. In our case, `bookstore-client` is the ID that represents our application in the Keycloak server.
- `spring.security.oauth2.client.registration.keycloak.client-secret`: This secret is used to authenticate the client with the Keycloak server. It's a confidential string known only to the application and the Keycloak server, acting as a password.
- `spring.security.oauth2.client.registration.keycloak.client-name`: A human-readable name for the client, which is `Keycloak` in our configuration.
- `spring.security.oauth2.client.registration.keycloak.provider`: Specifies the provider's name for this client registration. It's set to `keycloak`, linking this client registration to the Keycloak provider configured further down in the properties file.
- `spring.security.oauth2.client.registration.keycloak.scope`: Defines the scope of the access request. The `openid`, `profile`, and `email` scopes indicate that the application is requesting ID tokens and access to the user's profile and email information.
- `spring.security.oauth2.client.registration.keycloak.authorization-grant-type`: Specifies the OAuth2 flow to be used. Here, it's set to `authorization_code`, which is a secure and common method for obtaining access and refresh tokens.

- `spring.security.oauth2.client.registration.keycloak.redirect-uri`: This is the URI to which the user is redirected after logging in or out. `{baseUrl}` is a placeholder that Spring Security replaces with the application's base URL, ensuring that the redirect URI matches the application's domain.
- `spring.security.oauth2.client.provider.keycloak.issuer-uri`: This URL points to the Keycloak issuer URI for the realm you're using (`BookStoreRealm`), typically the base URL for Keycloak plus `/auth/realms/{realm-name}`. It tells the Spring Boot application where to find the Keycloak server for this realm.
- `spring.security.oauth2.resourceserver.jwt.issuer-uri`: Similar to the provider issuer URI, this setting configures the issuer URI for the JWT issuer. It is used by the resource server (your application) to validate JWTs. The issuer URI must match the issuer declared in the JWT for the token to be considered valid.

These settings wire our Spring Boot application to authenticate using Keycloak, specifying how our application should register with Keycloak, what scopes it requests, and how to validate tokens issued by Keycloak.

3. **Update the SecurityConfig file:** We need to update the `SecurityConfig` file to use the OAuth2 login with Keycloak:

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity
http) throws Exception {
    http
        .csrf(AbstractHttpConfigurer::disable)
        .sessionManagement(session -> session.
            sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(authz -> authz
            .requestMatchers("/login").permitAll()
            .anyRequest().authenticated()
        )
        .oauth2ResourceServer(oauth2 -> oauth2.
            jwt(Customizer.withDefaults()));

    return http.build();
}
```

Basically, in this code, we have only changed the last statement before `build` command.

`.oauth2ResourceServer(...)` configures OAuth2 resource server support and `.jwt(Customizer.withDefaults())` indicates that the resource server expects JWTs for authentication.

The latter uses the default JWT decoder configuration, which is suitable for most scenarios. This line is essential for integrating with OAuth2, where the application acts as a resource server that validates JWTs.

Also, we have excluded the `/login` endpoint from being secured because this endpoint should be public so that users can get credentials. This brings us to introducing a `LoginController` class with the `/login` endpoint.

4. **Introduce the `/login` endpoint:** We will need a `LoginRequestDto` class, which will be used as a body object for this POST endpoint, and a `LoginController` class. Let's write them as shown next.

This is the data transfer object from the client to the server. This object contains login credentials for the user created in the Keycloak server:

```
public record LoginRequestDto(String username, String password)
{ }
```

Also, we need one more configuration file to introduce the `RestTemplate` bean. We will use it in the `LoginController` class:

```
@Configuration
public class AppConfig {
    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

This is the `LoginController` class for introducing the `/login` endpoint to our application:

```
@RestController
public class LoginController {
    @Value("${spring.security.oauth2.client.registration.keycloak.client-id}")
    private String clientId;

    @Value("${spring.security.oauth2.client.registration.keycloak.client-secret}")
    private String clientSecret;

    @Value("${spring.security.oauth2.resourceserver.jwt.issuer-uri}")
    private String baseUrl;

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequestDto loginRequestDto) {
        String tokenUrl = baseUrl + "/protocol/openid-connect/
```

```
token";

        // Prepare the request body
        MultiValueMap<String, String> requestBody = new
LinkedMultiValueMap<>();
        requestBody.add("client_id", clientId);
        requestBody.add("username", loginRequestDto.username());
        requestBody.add("password", loginRequestDto.password());
        requestBody.add("grant_type", "password");
        requestBody.add("client_secret", clientSecret);

        // Use RestTemplate to send the request
        RestTemplate restTemplate = new RestTemplate();
        ResponseEntity<String> response = restTemplate.
postForEntity(tokenUrl, requestBody, String.class);

        // Return the response from Keycloak
        return ResponseEntity.ok(response.getBody());
    }
}
```

Let's understand what this controller does when the user sends a POST request to the / login endpoint.

This is the first time we are using the @Value annotation of Spring Boot. This annotation injects property values into fields. Here, it's used to inject the Keycloak client ID, client secret, and URL of the Keycloak server from the application's properties file into the `clientId` variable.

Our application basically gets the username and password of the user from the request body and prepares a REST call to the Keycloak server with the parameters in the application properties file. It gets the response from the Keycloak server and returns the response to the user. In order to demonstrate what it does, please see the following figure, *Figure 5.5*.

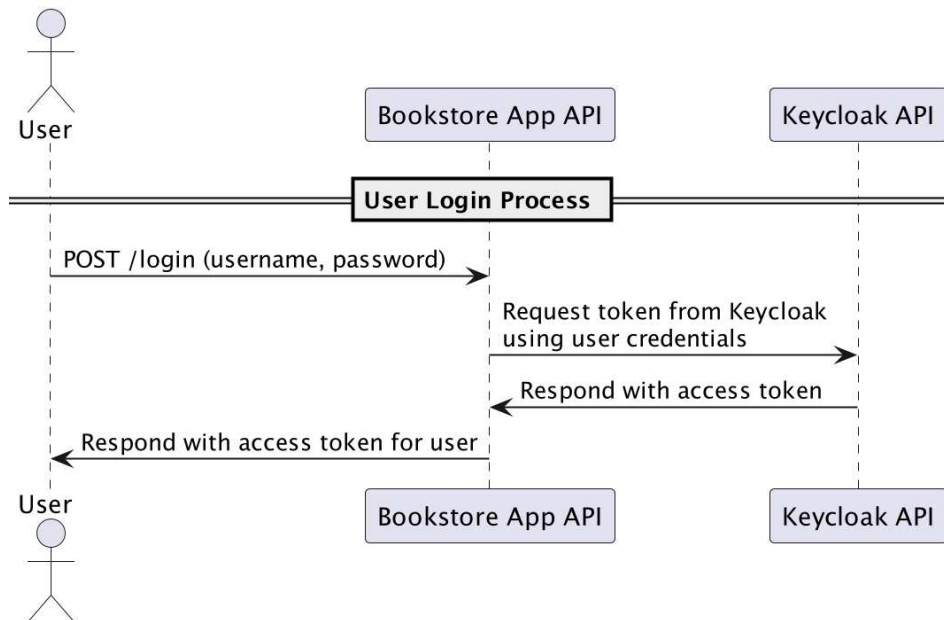


Figure 5.5: User login process with Keycloak server

As shown in the diagram, the user makes a call. Our application prepares the requests, makes a POST call to the Keycloak API, and returns the response to the user. We will use this access token in our calls to our application. We will test this out in the next section.

Testing our endpoints with an access token

Let's run our application. As you'll remember, in our previous tests, we received the HTTP 403 Forbidden message from our application. Now we will test it after we get the access token from the login process:

```
curl --location "http://localhost:8080/login"
--header "Content-Type: application/json"
--data "{
  \"username\":<username>,
  \"password\":<password>
}"
```

Please change the username and password values to those of the user you have created in the Keycloak server. You will get a response like the following:

```
{
  "access_token": <JWT Token>
  "expires_in": 300,
  "refresh_expires_in": 1800,
  "refresh_token": <JWT Token>,
  "token_type": "Bearer",
  "not-before-policy": 0,
  "session_state": "043d9823-7ef4-4778-b746-10dd8e75baa4",
  "scope": "email profile"
}
```

I haven't put the exact JWT here since it is a huge alphanumeric value. Let's explain what these values are:

- `access_token`: This is a JWT that the client application can use to access protected resources by passing it in the authorization header of HTTP requests. It is encoded and contains claims (or assertions) about the authentication and authorization of the user. The token itself is opaque to the client but can be decoded and verified by the resource server (or any party with the appropriate key).
- `expires_in`: Specifies the lifetime of the access token in seconds. After this time, the access token will no longer be valid for accessing protected resources. In this example, the access token expires in 300 seconds (5 minutes).
- `refresh_expires_in`: Indicates the lifetime of the refresh token in seconds. The refresh token can be used to obtain a new access token when the current access token expires. Here, it is set to 1800 seconds (30 minutes).
- `refresh_token`: This is another JWT, similar to the access token but used solely for obtaining new access tokens without requiring the user to log in again. It has a longer validity period than the access token and should be stored securely.
- `token_type`: Specifies the type of token issued. In OAuth2, this is typically `Bearer`, which means that the bearer of this token is authorized to access the resources. The client application should use this token type when constructing the authorization header for HTTP requests.
- `not-before-policy`: This field is specific to Keycloak and similar authorization servers. It indicates a policy or timestamp before which the token should not be considered valid. A value of 0 typically means the token is valid immediately upon issuance.

- `session_state`: A unique identifier for the user session that the token is associated with. This can be used by the application for session management or tracking purposes.
- `scope`: Specifies the scope of the access requested. Scopes are space-delimited and indicate what access rights the application has been granted. In this case, `email profile` means the application can access the user's email address and profile information.

Now, we will use this access token and use it in the header of our requests:

```
curl --location "http://localhost:8080/books"  
--header "Authorization: Bearer <access_token>"
```

We will get a list of books as a response. You can also use the same token to make requests for the other endpoints. After 5 minutes (300 seconds), the access token will expire, and we need to call the `/login` endpoint one more time to gather a new access token.

We have finally come to the end of this section. We ran the Keycloak server locally and defined a new realm, client, and user. Later, we configured our Spring Boot application to communicate with the Keycloak server. After all these configurations and implementations, we could gather an access token with our username and password and get a valid response from our protected endpoints.

In the next section, we will learn how to define a role and filter the role of the request by role; this is an important step to protect our endpoints with role-based security.

Implementing RBAC in Spring Boot

Making sure that users only get access to the resources that they are entitled to is of paramount importance in the landscape of modern web development. This is where RBAC comes in. Imagine setting up a series of gates within your application, each requiring a specific key that only certain users possess. This is the essence of RBAC—ensuring that access is granted based on the roles assigned to a user, enhancing both security and usability.

Why prioritize RBAC in your Spring Boot application with Keycloak? Well, first of all, it simplifies the complex task of access management, making it easier for developers to define and enforce security policies. This allows your applications to tap into Keycloak's great support for OAuth2 and provides a very structured, scalable way to secure endpoints. This will make your application more secure and its features clearer when it comes to controlling access to a user. As we delve more into setting up RBAC with Keycloak, remember that this isn't about restriction per se; it is more about seamless and secure experiences for your user, meaning they get the right tools and permissions to navigate through your application with efficacy. Let's embark on this journey of bringing out the full potential of role-based security within our Spring Boot applications.

Defining roles and permissions in Keycloak

Defining roles and permissions in Keycloak is important for establishing secure applications that offer the management of user access in a very streamlined manner. Through this process, you will be able to specifically outline what a certain user can perform and thus enhance security and productivity in the system. Here's a straightforward guide to setting up roles and permissions in Keycloak, along with insights into what kind of configuration this brings about in terms of your security and management:

1. First, log in to the Keycloak Admin Console (<http://localhost:8180>) using your administrator credentials. This is our control panel for managing realms, users, roles, and permissions.
2. Navigate to the **Roles** section—select the realm you wish to configure from the drop-down menu, then click on **Roles** in the left-hand menu. Here, you'll see a list of existing roles.

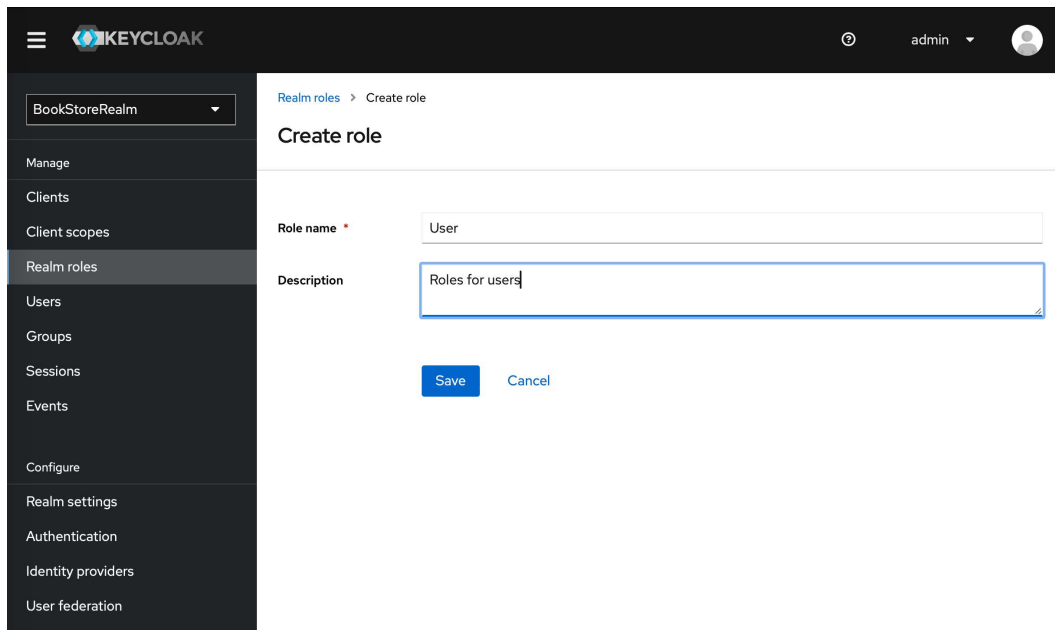


Figure 5.6: Add Role screen

- I. **Add roles:** Click on **Add Role**. Enter a name for the user role and a description that helps you identify the role's purpose within your application.
 - II. **Save:** Click **Save**. You've now created a role that can be assigned to users.
 - III. Add one more role called `admin`.
3. Go to **Users**, select the user you created in the previous step and click on **Role Mappings**.

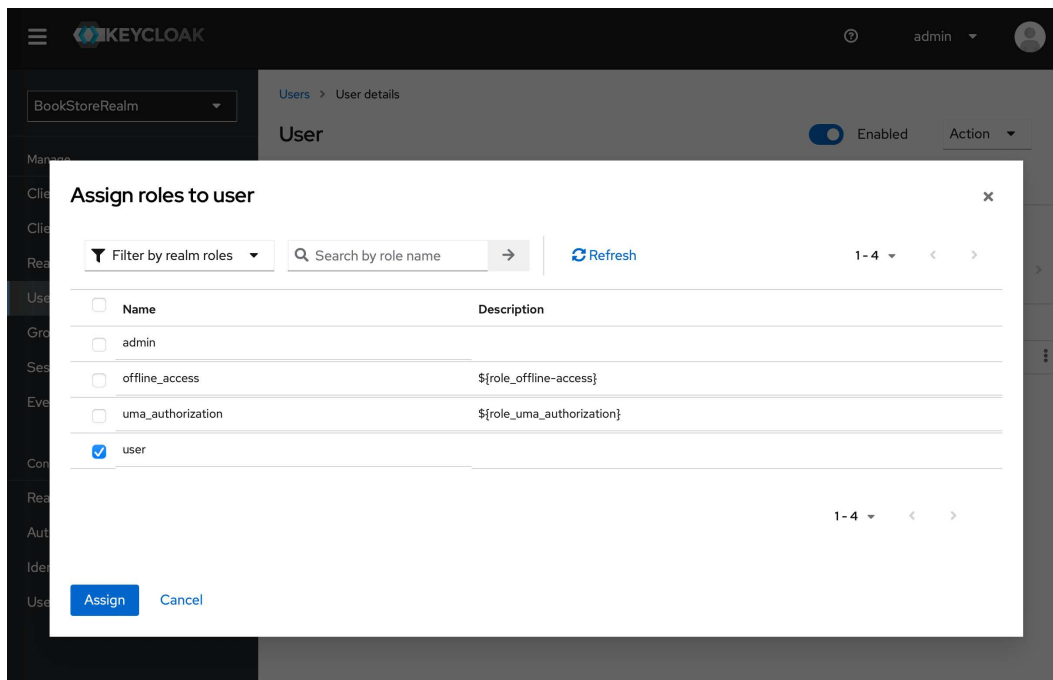


Figure 5.7: Role Mappings screen of the user

- I. Here, you can assign the **User** role to the user. Select the role and click **Assign**.
- II. Create a new user and assign the **admin** role to this user.

With the roles defined and assigned, let's understand the process of including the roles we created in the application.

Tailoring the book store application for role-based access

Implementing RBAC in your Spring Boot book store application using Keycloak significantly enhances its security, ensuring that users can access only what they are permitted to according to their roles. This not only makes your application more secure by design but it also sets up a very solid framework for the management of user permissions. Let's follow the steps to include this role-based setup in your application, wherein a new class—`KeycloakRoleConverter`—is introduced and specific security configurations. The new class will be an adapter between roles from Keycloak and roles in Spring Security. Let's learn step by step how to implement this structure into our application:

1. **Update security configurations:** Start by defining the security filter chain in our Spring Boot application. This involves specifying which endpoints require what level of authority. Here, we introduce two major changes: restricting POST requests to `/books` and `/authors` to users with the `ROLE_ADMIN` authority and configuring the OAuth2 resource server to use a custom JWT authentication converter. Open your security configuration class and update the `securityFilterChain` bean as follows:

```
@EnableMethodSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity
http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(session -> session.
sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(authz -> authz
                .requestMatchers("/login").permitAll()
                .requestMatchers(HttpMethod.POST, "/books", "/"
authors").hasAuthority("ROLE_ADMIN")

                .requestMatchers(HttpMethod.GET, "/books/**", "/reviews/**",
"/authors/**", "/publishers/**").hasAnyAuthority("ROLE_
USER", "ROLE_ADMIN")
                .anyRequest().authenticated()
            )
            .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt ->
jwt.jwtAuthenticationConverter(new KeycloakRoleConverter())));
        return http.build();
    }
}
```

2. **Implement the KeycloakRoleConverter class:** This class is pivotal as it translates Keycloak JWTs into Spring Security's authentication structure. This custom converter extracts roles from the JWT and assigns them as authorities within the Spring Security context:

```
public class KeycloakRoleConverter implements Converter<Jwt,
AbstractAuthenticationToken> {
    @Override
    public AbstractAuthenticationToken convert(Jwt jwt) {
        // Default converter for scopes/authorities
        JwtGrantedAuthoritiesConverter
defaultAuthoritiesConverter = new
JwtGrantedAuthoritiesConverter();
        Collection<GrantedAuthority> defaultAuthorities =
defaultAuthoritiesConverter.convert(jwt);

        // Extract realm_access roles and map them to
GrantedAuthority objects
        Collection<GrantedAuthority> realmAccessRoles =
extractRealmAccessRoles(jwt);

        // Combine authorities
        Set<GrantedAuthority> combinedAuthorities = new
HashSet<>();
        combinedAuthorities.addAll(defaultAuthorities);
        combinedAuthorities.addAll(realmAccessRoles);

        return new
AbstractAuthenticationToken(combinedAuthorities) {
            @Override
            public Object getCredentials() {
                return null;
            }

            @Override
            public Object getPrincipal() {
                return jwt.getSubject();
            }
        };
    }

    public static List<GrantedAuthority>
extractRealmAccessRoles(Jwt jwt) {
        Map<String, Object> realmAccess = jwt.
getClaimAsMap("realm_access");
        if (realmAccess == null) {
            return Collections.emptyList();
        }
    }
}
```

```
    }

    List<String> roles = (List<String>) realmAccess.
get("roles");
    if (roles == null) {
        return Collections.emptyList();
    }

    return roles.stream()
        .map(roleName -> new
SimpleGrantedAuthority("ROLE_" + roleName.toUpperCase()))
        .collect(Collectors.toList());
    }
}
```

We have introduced a class that implements the `Converter` interface to convert a JWT into an `AbstractAuthenticationToken` class, a concept used in Spring Security for authentication information. With those changes made, our book store application now has a very secure and powerful RBAC system in place. It only lets an authenticated user with the right role do a certain action—this significantly increases the security and integrity of your application. Also, that setup is granular, and it affords the best control of user permissions and eases the management of access rights across our application.

We can test our app now. When we log in using the `user` role and make a GET request to `/books`, we will get a successful response, but when we try to make the POST request to `/books` and create a new book, the response will be `forbidden`. If we log in with the `admin` role, we will always have success, in the case of a request having either GET or POST. Securing a foothold in this area of RBAC for our Spring Boot book store application will open up a way for expanding the security architecture very favorably.

The next section of our journey is into securing reactive applications. Moving forward, this chapter will address how all the principles of security, authentication, and authorization apply to a reactive context of programming. This will not only help us to gain an expanded scope of understanding the approaches to be utilized regarding security in practice but also equip us with the tools that will help us to offer adequate protection of our reactive applications. We will learn about the depths of reactive security to give our application the resilience needed to meet evolving cyber threats.

Securing reactive applications

In the dynamic landscape of software development, securing reactive applications brings with it a new set of challenges and opportunities. Having dived deeper into the world of reactive programming, now we need to tweak our security strategies to align with the non-blocking, event-driven nature of these applications. Reactive systems that are identified to handle a large number of concurrent data

streams strongly call for a strong yet flexible security approach. This section will try to unfold the complications in securing reactive applications and will guide you through the essential steps and considerations in order to effectively protect your reactive ecosystem.

We will look into how to use the reactive support provided by Spring Security to enable those security features without breaking the reactive principles, ensuring high responsiveness and resilience.

Transitioning to the reactive programming territory could mean a challenge in terms of security, especially with the full strength Spring Security has to offer. Implementing reactive security with Spring Security is actually about adapting classic security paradigms to work within this asynchronous, non-blocking model of a reactive application. That realignment could be described as a shift, not one of any technical change but one in how security processes interact with data flows and user requests. The security model is required to be functional in this environment, without seeking any compromises with the reactive principles or imposing bottlenecks on it.

One of the key differences in securing reactive applications lies in the way authentication and authorization are done. Unlike traditional servlet-based applications where usually security contexts are tied to a thread-local, reactive security has to be able to handle this type of decoupled, stateless nature of reactive programming. Spring Security provides a reactive security context that gets scoped to the reactive stream, ensuring that decisions about security in a context-aware way are aligned with the flow of the application.

In this series, when we look into the reactive part of Spring Security, we will see how to include them effectively within our applications for the reactive system's security. This covers handling the reactive APIs offered by Spring Security, understanding who acts as a publisher for making security decisions (Mono and Flux), and making sure your application stays secure, reactive, and scalable. This guide, with practical examples and a step-by-step approach, will help you navigate the complexities of reactive security to ensure the application is not only secure but also performs and scales to the expectations laid down by the reactive programming model.

Let's start implementing security into the reactive application. We will use the same project we developed in *Chapter 3*:

1. **Add dependencies:** Ensure your Spring Boot project has the following dependencies in the `build.gradle` file:

```
implementation 'org.springframework.security:spring-security-  
config'  
implementation 'org.springframework.boot:spring-boot-starter-  
oauth2-client'  
implementation 'org.springframework.boot:spring-boot-starter-  
oauth2-resource-server'
```

2. **Configure** `application.properties`: Add the following properties to your application. properties file, replacing placeholders with your actual Keycloak and client details:

```
spring.security.oauth2.client.registration.keycloak.client-id=bookstore-client
spring.security.oauth2.client.registration.keycloak.client-secret=<Your-Client-Secret>
spring.security.oauth2.client.registration.keycloak.client-name=Keycloak
spring.security.oauth2.client.registration.keycloak.provider=keycloak
spring.security.oauth2.client.registration.keycloak.scope=openid,profile,email
spring.security.oauth2.client.registration.keycloak.authorization-grant-type=authorization_code
spring.security.oauth2.client.registration.keycloak.redirect-uri={baseUrl}/login/oauth2/code/keycloak
spring.security.oauth2.client.provider.keycloak.issuer-uri=http://localhost:8180/auth/realms/BookStoreRealm
spring.security.oauth2.resourceserver.jwt.issuer-uri=http://localhost:8180/auth/realms/BookStoreRealm
```

3. **Define** a `SecurityWebFilterChain` class in your `SecurityConfig` class to specify authorization rules and set up the JWT converter for role extraction:

```
@Configuration
public class SecurityConfig {

    @Bean
    public SecurityWebFilterChain
    securityWebFilterChain(ServerHttpSecurity http) {
        http
            .csrf(ServerHttpSecurity.CsrfSpec::disable)
            .authorizeExchange(exchanges -> exchanges
                .pathMatchers("/login").permitAll()
                .pathMatchers(HttpMethod.POST, "/users").
                hasAuthority("ROLE_ADMIN")
                .pathMatchers(HttpMethod.GET, "/users/**").
                hasAnyAuthority("ROLE_ADMIN", "ROLE_USER")
                .anyExchange().authenticated()
            )
            .oauth2ResourceServer(oauth2ResourceServer ->
                oauth2ResourceServer.jwt(jwt ->
                    jwt.
                    jwtAuthenticationConverter(jwtAuthenticationConverter()))
            );
    }
}
```

```

        return http.build();
    }

    private Converter<Jwt, ? extends Mono<? extends
AbstractAuthenticationToken>> jwtAuthenticationConverter() {
        ReactiveJwtAuthenticationConverter jwtConverter = new
ReactiveJwtAuthenticationConverter();
        jwtConverter.setJwtGrantedAuthoritiesConverter(new
KeycloakRoleConverter());
        return jwtConverter;
    }
}

```

4. **Define KeycloakRoleConverter:** The KeycloakRoleConverter class is essential for mapping Keycloak roles to Spring Security authorities:

```

public class KeycloakRoleConverter implements Converter<Jwt,
Flux<GrantedAuthority>> {

    @Override
    public Flux<GrantedAuthority> convert(final Jwt jwt) {
        // Extracting roles from realm_access
        return Flux.fromIterable(getRolesFromToken(jwt))
            .map(roleName -> "ROLE_" + roleName.
toUpperCase()) // Prefixing role with ROLE_
            .map(SimpleGrantedAuthority::new);
    }

    private List<String> getRolesFromToken(Jwt jwt) {
        Map<String, Object> realmAccess = jwt.
getClaimAsMap("realm_access");
        if (realmAccess == null) {
            return Collections.emptyList();
        }

        List<String> roles = (List<String>) realmAccess.
get("roles");
        if (roles == null) {
            return Collections.emptyList();
        }

        return roles;
    }
}

```

5. **Introduce the token endpoint:** Lastly, create a `LoginController` class to handle authentication requests, utilizing Keycloak's token endpoint:

```
@RestController
public class LoginController {

    @Value("${spring.security.oauth2.client.registration.
keycloak.client-id}")
    private String clientId;

    @Value("${spring.security.oauth2.client.registration.
keycloak.client-secret}")
    private String clientSecret;

    @Value("${spring.security.oauth2.resourceserver.jwt.issuer-
uri}")
    private String baseUrl;

    @PostMapping("/login")
    public Mono<ResponseEntity<?>> login(@RequestBody
LoginRequestDto loginRequestDto) {
        // URL for Keycloak token endpoint
        String tokenUrl = baseUrl + "/protocol/openid-connect/
token";

        // Prepare the request body
        MultiValueMap<String, String> requestBody = new
LinkedMultiValueMap<>();
        requestBody.add("client_id", clientId);
        requestBody.add("username", loginRequestDto.username());
        requestBody.add("password", loginRequestDto.password());
        requestBody.add("grant_type", "password");
        requestBody.add("client_secret", clientSecret);

        // Use RestTemplate to send the request
        RestTemplate restTemplate = new RestTemplate();
        ResponseEntity<String> response = restTemplate.
postForEntity(tokenUrl, requestBody, String.class);

        // Return the response from Keycloak
        return Mono.just(ResponseEntity.ok(response.getBody()));
    }
}
```

Now we've introduced role-based security filters into our reactive application. Almost everything is very similar to traditional applications, only the responses are in the reactive realm, such as `Mono` and `Flux`. We can use our previous `curl` scripts for the login to get an access token for both `user` and `admin` roles, and we can test our `POST` and `GET` endpoints.

It means that integrating reactive security into a Spring Boot application requires deliberately configuring a sequence of custom implementations with Spring Security especially using OAuth2 with Keycloak. From the setup of dependencies and the configuration of properties to the security filter chain, ending in a custom role converter—every step has been described so that they can be followed to get a secure reactive environment for our book store application. This implementation not only leverages the non-blocking nature of reactive programming but also ensures that our application is secure and scalable for it to be able to handle the demand proficiently in a reactive context.

Summary

With this, we shall finish this chapter, which dealt with securing Spring Boot applications. We have now finished our journey through Spring Boot security by learning about the context and the toolsets that are required to secure our applications in an effective manner. Let's summarize the key learnings from this chapter:

- **Understanding Spring Boot security:** We understood the need for securing our Spring Boot applications and the basic tenets of Spring Security.
- **Implementing with OAuth2:** We learned how to authenticate users using OAuth2 and manage secure tokens using JWT.
- **RBAC using Keycloak:** We showed in great detail how to configure Keycloak to manage roles and permissions in our system, thus enhancing the security structure of our application.
- **Modified security configuration for reactive:** We elaborated on how security configurations are customized for the reactive programming model so that our applications can be both secure and capable at the same time.
- **Reactive security with Spring Security implementation:** In order to implement security in a reactive environment, crucial differences and modifications are necessary. This chapter emphasized non-blocking and event-driven security mechanisms.

This chapter gave a brief outline of what Spring Security is about. The next chapter, Advanced Testing Strategies, drills further into the Spring Boot ecosystem, this time getting into the depths of testing. The chapter will delve more into the differences between unit and integration testing, which will point out the challenges with testing reactive components, leading to a discussion of security features testing with a short introduction on the aspect of **Test-Driven Development (TDD)** with Spring Boot 3.0. This progression from securing to testing our applications really underscores the comprehensive approach that needs to be taken for developing resilient, high-quality software ready to meet the demands of modern application development.

